

Activity 04: Evaluation

SISMID Lab

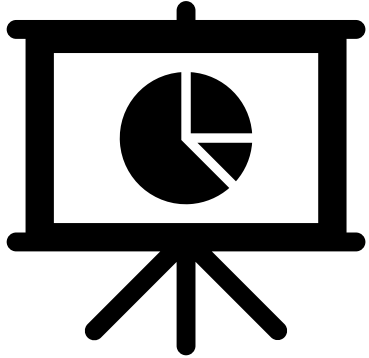
July 22 through July 24, 2026

A Human – AI Teaming Approach to Infectious Disease Modeling

Lecture Overview

Topic	Slides
1. General Housekeeping	3
2. Lab Timeline	4
3. Task Overviews	5
4. Goal Output	6
5. Evaluation Metrics	7–14
6. Scoring with scoringutils	15–18
7. Set-Up and Overview	19–20
8. Task 1: Input Data	21–23
9. Task 2: Scoring the Forecast	24–27
10. Task 3: Output Table	28–30
11. Task 4: Summary by Horizon	31–33

General Housekeeping

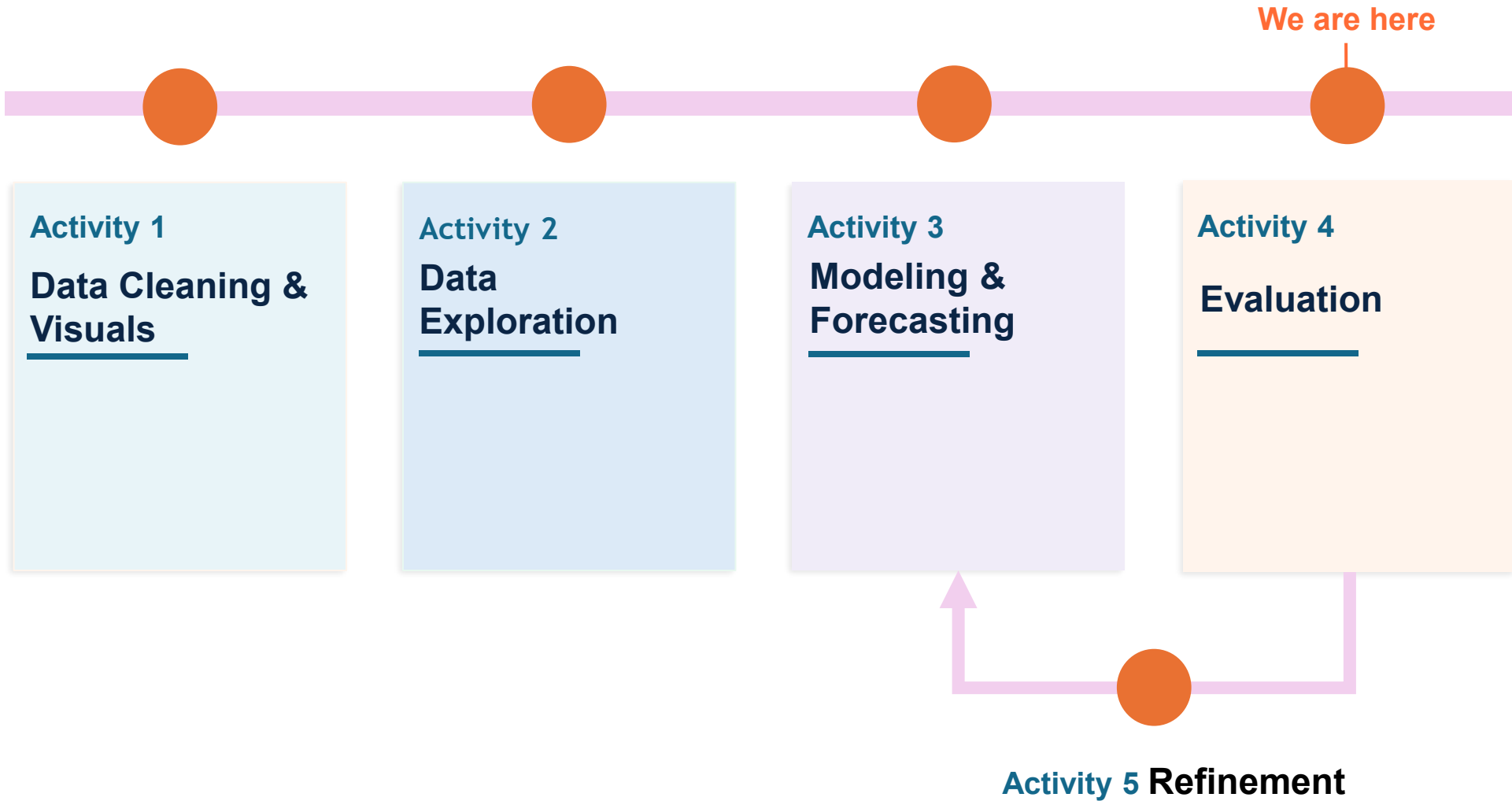


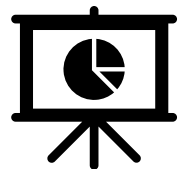
Any slide with this icon contains the **overview** and **expected outputs** for the included tasks. All outputs listed on these slides should be uploaded to the course GitHub repository for review.



Any slide with this icon indicates that it is **time to complete a hands-on activity**. Carefully follow the instructions provided, complete the required task, and verify your results.

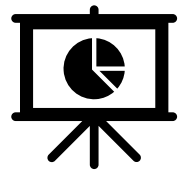
Lab Timeline





Task Overviews

Task	Description
Task 0: Set-Up	Add the section headers and file paths and set a global rule telling the AI to use the <i>scoringutils</i> package for the entire pipeline and not to hand-code the scoring formulas.
Task 1: Input Data	Load the FluSight forecast file and the cleaned observed data, join them on a common key (<code>target_end_date</code> and <code>week</code>), and drop the targets that have no observed value to score against.
Task 2: Scoring the Forecast	Rename the joined columns to the <i>scoringutils</i> conventions and add a model label, build the forecast object with <code>as_forecast_quantile()</code> , score it, and extract WIS, AE, and 95% PI coverage.
Task 3: Output Table	Format the first output table, one row per reference date and horizon, sorted ascending and rounded to one decimal, then print it and save it to <code>output/data/04_evaluation/</code> .
Task 4: Summary by Horizon	Average the per-forecast scores across reference dates within each horizon to show how forecast skill changes with lead time, then print and save the second table.



Goal Output

From the activity (see previous slide), you should produce two scored output tables. Guided by the rules you write, an AI agent scores your Activity 3 forecasts against the cleaned admissions data using the `scoringutils` package, and saves the results through `scripts/04_evaluation.R`.

Activity Output	Description
forecast_scores.csv	Per-forecast scores, one row per reference date and horizon, with columns for <code>reference_date</code> , <code>horizon</code> , <code>target_end_date</code> , <code>observed</code> , <code>WIS</code> , <code>AE</code> , and <code>coverage_95</code> .
forecast_scores_by_horizon.csv	Summary scores by horizon, with <code>n</code> plus the mean <code>WIS</code> , <code>MAE</code> , and 95% coverage and the minimum and maximum of <code>WIS</code> and <code>MAE</code> , showing how forecast skill decays with lead time.

Evaluation Metrics

Overview

- We are going to focus on calculating some of the primary metrics used by the **FluSight** challenge, and commonly throughout forecast evaluation
- This includes, but is not limited to, mean absolute error (MAE), 95% prediction interval (PI) coverage, and weighted interval score (WIS)
- Note that the metrics come with their own **strengths** and **weaknesses**, and represent only a **minuscule** portion of the evaluation approaches available

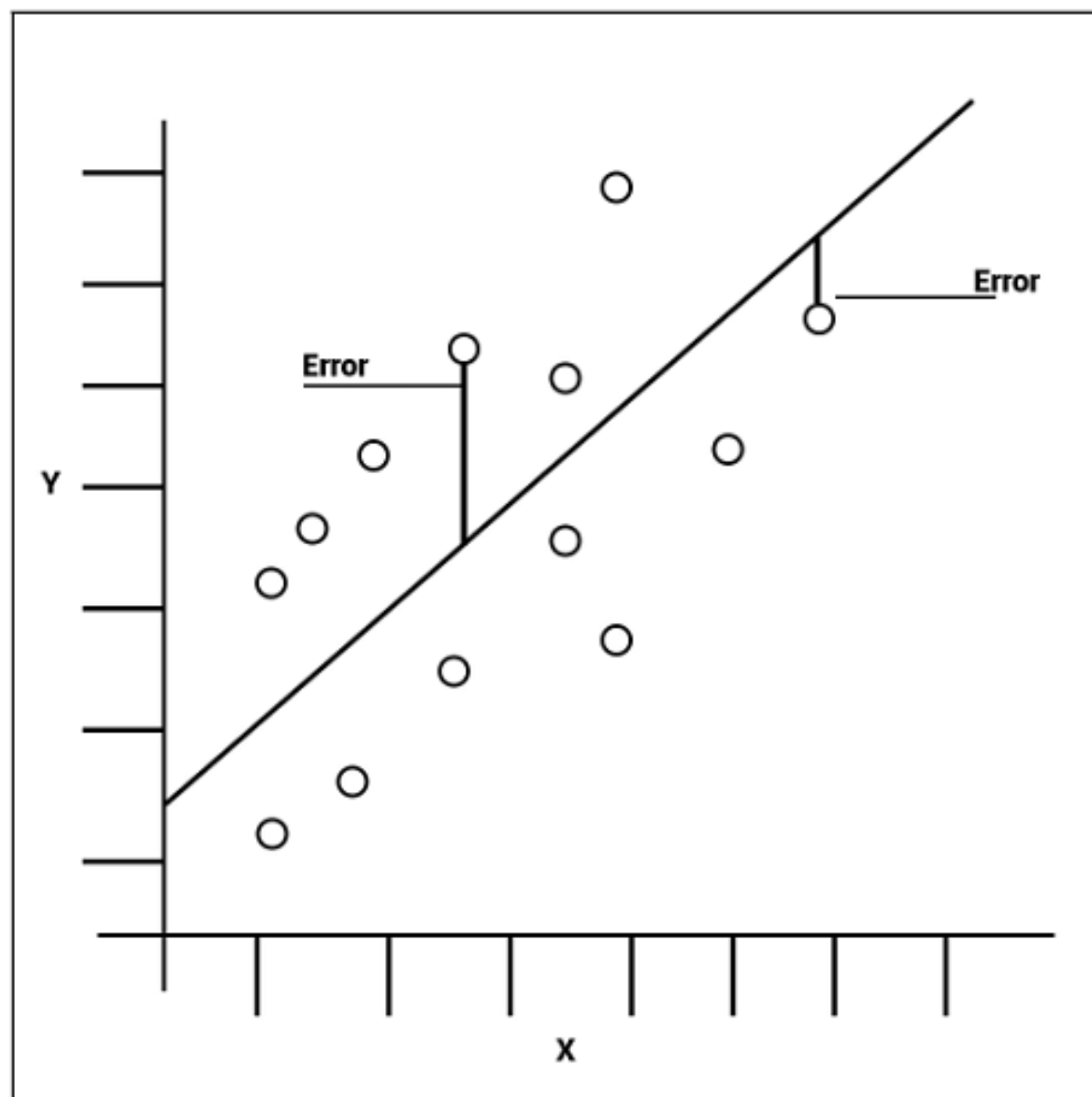
Mean Absolute Error (MAE)

- Measure of how **accurate** point (single value per time point) forecasts are (i.e., how far the model's predictions fall from the observed admission counts).

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

N	Number of observations
y_i	Observed value
$\hat{y}_i$	Predicted value (e.g., median forecast value)

Ideal Value: Lower is better!



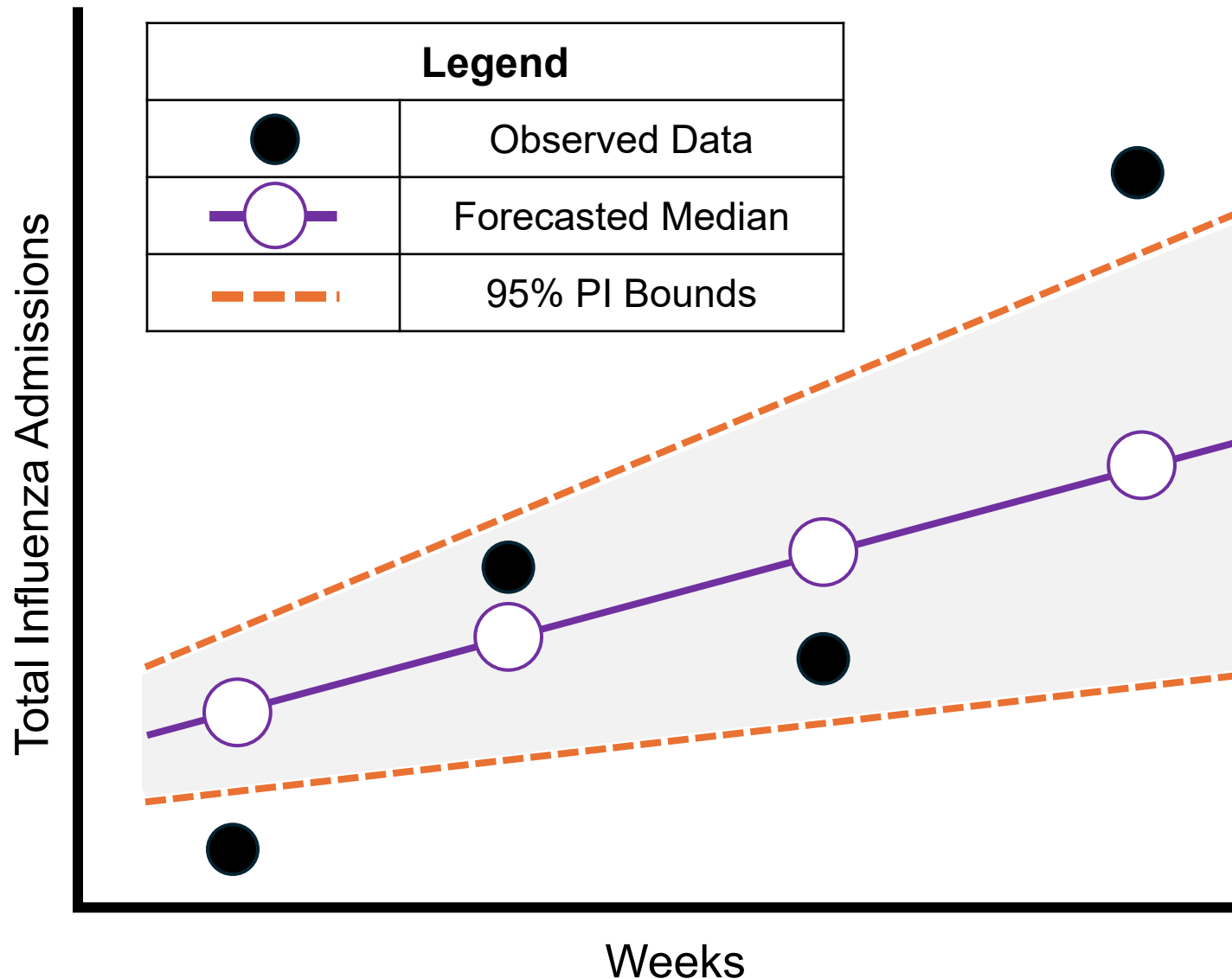
95% Prediction Interval (PI) Coverage

- Measures how often the observed value is captured within the bounds of the 95% prediction interval (i.e., measure of how well the **model captures uncertainty**).

$$95\% \text{ PI Coverage} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(l_i \leq y_i \leq u_i)$$

N	Number of observations
$\mathbf{1}(\cdot)$	Indicator (1 if the observation falls in the interval, 0 otherwise)
y_i	Observed value
l_i, u_i	Lower/Upper bounds of the 95% PI

Ideal Value: ~95%; Below 95% = intervals too narrow (overconfident);
above 95% = too wide (underconfident).



Calculating the 95% PI Coverage

1. We are working with four dates of data ($N = 4$).
2. For 2 of those 4 dates, the observed data falls within the bounds.

$$\frac{\# \text{ Observed data dates within bounds}}{\text{Total Number of Dates } (N)} = \frac{2}{4} = 0.5 \text{ or } 50\% \text{ PI Coverage}$$

50% is far below the nominal 95%, so these intervals are too narrow — the model is **overconfident**.

Interval Score (IS)

- Measures how well a **single** prediction interval performs, rewarding intervals that are narrow but penalizing ones that the observed value falls outside, in the units of the observed data.

$$IS_{\alpha}(F, y) = \underbrace{(u - l)}_{\text{Dispersion}} + \underbrace{\frac{2}{\alpha} * (l - y) * \mathbf{1}(y \leq l)}_{\text{Overprediction}} + \underbrace{\frac{2}{\alpha} (y - u) * \mathbf{1}(y \geq u)}_{\text{Underprediction}}$$

l, u	Lower/Upper bounds of prediction interval
y	Observed value
$\mathbf{1}(\cdot)$	Equals 1 when the stated condition is TRUE, 0 otherwise

Ideal Value: Lower is better!

Weighted Interval Score (WIS)

- Measures overall forecast quality, focusing on how close the whole predicted distribution is to the observed value, in the units of the observed data. It rewards forecasts that are both **accurate** and **appropriately confident**.

$$WIS = \frac{1}{K + 0.5} * \left(0.5 * |y - m| + \sum_{k=1}^K w_k * IS_{\alpha_k}(F, y) \right)$$

K	Number of Prediction Intervals
y	Observed value
m	Forecasted Median
w_k	Weight Assigned to each interval; $w_k = \frac{\alpha_k}{2}$
$IS_{\alpha_k}(F, y)$	Interval score

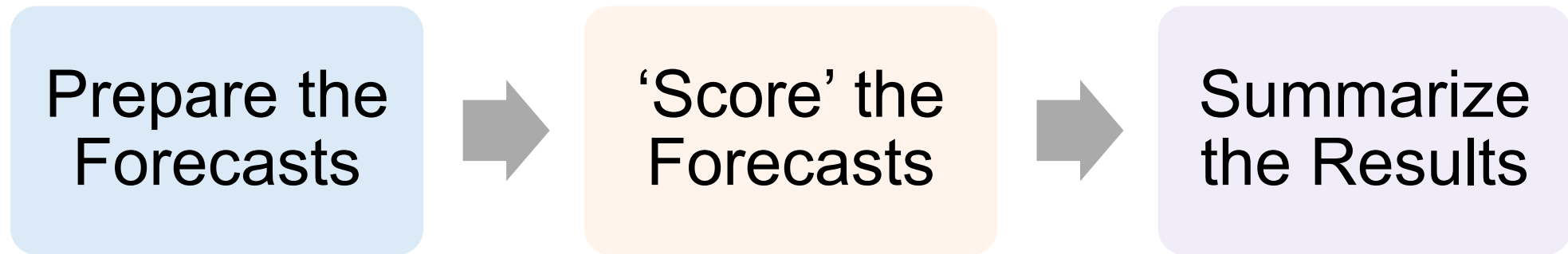
Ideal Value: Lower is better!

scoringutils Package

What should we use to calculate the metrics?

From Formulas to Code

- **scoringutils** is an R package that provides multiple built-in metrics and proper scoring rules to simplify the process of evaluating probabilistic forecasts against true observed values.
- It's easy to implement (three steps):



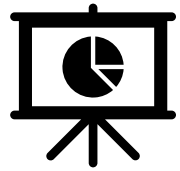
- We will work through each step together (and with AI) to **evaluate our ARIMA models** from the last activity.

Required Data Format

- It expects at least `observed`, `predicted`, and `quantile_level` columns. However, it can take 'grouping' variables as well (e.g., `location`, `horizon`) that can be used to group and summarize scores.
- Our current set-up maps to the package requirements as follows:

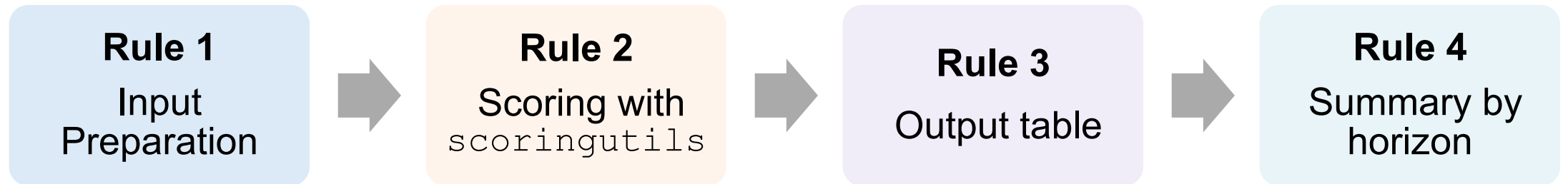
Required Package Format	In Hubverse terms	Description
<code>forecast_unit</code>	<code>location</code> , <code>reference_date</code> , <code>horizon</code> , <code>target_end_date</code> , <code>model</code>	The columns that uniquely identify one forecast; also used for later summarization.
<code>observed</code>	<code>cleaned_flu_admissions.csv</code>	The observed truth; we will join the truth data with the forecast file.
<code>predicted</code>	<code>value</code>	The forecasted value.
<code>quantile_level</code>	<code>output_type_id</code>	The quantile the value corresponds to (0 to 1).

Implementation



Overview

- Rather than hand-coding the evaluation, we'll write rules in *rules.md* that an AI uses to build the script (*04_evaluation.R*), covering the whole pipeline, from preparing the data to writing the output tables.
- Our end goal is two tables: one with forecast-by-forecast performance (Rule 3), and one with summary metrics by horizon (Rule 4).





Initial Set-Up

- Prior to specifying Rules 1 - 4 for evaluating forecasts, we need to do some initial set-up like we have done for other activities (e.g., headers, file paths, etc.).
- Given we specifically want to use the `scoringutils` package for the entire pipeline, we want to make sure this is clear and respected by the AI. Therefore, we are going to add a global rule specifying this.

Forecast Evaluation

Save all code in this section to ``output/scripts/04_evaluation.R``. If the folder path is not present, create it before saving the script.

Use the **`**scoringutils**`** package to compute the metrics – do not hand-code the scoring formulas.

Loading the Data (## 1. Input Data)

- We need to be **explicit** about what files we want to include as part of the evaluation.
- `scoringutils` requires both forecasts and observed data in a single data frame; we need **to load the files** prior to merging them.

File Type	Forecasts	Observed Truth
File Path	output/data/03_forecast/flusight_forecasts.csv	output/data/01_cleaning/cleaned_flu_admissions.csv
File Format	reference_date, target, horizon, target_end_date, location, output_type, output_type_id, value	week, location, value

Joining Forecasts & Truth Data

- We now need to join each forecast row to the observed truth by matching on a **common variable** (Hint: It's `target_end_date` & `week`).
- Only forecasts with a **matching observed value can be scored**. The 2- and 3-week-ahead targets that fall beyond the last observed week have no truth to compare against, so we drop them.
- **Including a message** of *how many combinations were dropped* for lack of an observed target will act as a check in the code (i.e., is it functioning as it should).



Now it's your turn!

On your own, use the previous two slides to specify rules to load the proper data files and merge them on a common key (**Hint:** `target_end_date & week`)

- 1 **Create** the ## 1. Input Data section header under the **initial set-up code** in **rules.md**.
- 2 **Write detailed rules** to load the two needed files (**Hint:** provide the file paths for the forecasts & observed data), merge them on a common key, and drop targets with no observed value.
- 3 Regenerate **agents.md** from your updated rules document.
- 4 Use the **agents.md** file and Copilot (or your preferred AI) to generate **scripts/04_evaluation.R**. Run the script and check the output.

Shaping the Data (## 2. Scoring with scoringutils)

- `scoringutils` only recognizes its own column names, so before scoring we **relabel the joined data** to match its conventions.
- The FluSight data is already long (one row per quantile), so this is a rename, not a reshape. **No pivoting needed**; we're relabeling columns, not restructuring rows.
- **Three renames**: `output_type_id` → `quantile_level`, `value` → `predicted`, and the joined truth value → `observed`.
- **Add a constant model column** (e.g. "arima"). `scoringutils` is built to compare models, so it expects a model label; with only one model we still include it, since it's part of the *forecast unit*.

Scoring the Forecast

- Prior to scoring the forecast (i.e., evaluating the forecasts), we convert our new data frame to a **forecast object** (data table with class of forecast).
- `scoringutils::as_forecast_quantile()` creates the forecast object; we **specify the forecast unit** (what defines a unique forecast). This allows each `reference_date × horizon` combination to be evaluated as its own forecast.
 - **Forecast unit:** `reference_date, horizon, target_end_date, location, model`
- Finally, we call `scoringutils::score()` on our forecast object.

Extracting the Metrics

- `scoringutils` provides a **plethora of metrics**. We can provide rules to make sure that the AI extracts (or calculates) only the metrics we want:

WIS: The `wis` column returned by `score()`.

AE: the `ae_median` column (absolute error of the median); averaged across reference dates it becomes MAE (Rule 4).

95% PI coverage: The 95% interval coverage metric.

- As a precaution, we can give back up options (think substitute options) in case a metric **cannot be extracted**. For example, for 95% PI coverage:

If `score()`'s defaults do not already include it, add it via `scoringutils`' interval-coverage metric at the 95% range (e.g. supply a custom `interval_coverage` metric with the interval range set to 95).



Now it's your turn!

We will work through the following steps together to add the next section to our **rules.md** document.

- 1 **Create** the `## 2. Scoring with scoringutils` section header in **rules.md**.
- 2 **Write detailed rules** to shape the data, score the forecast, and extract the metrics of interest (e.g., AE, WIS, 95% PI coverage).
- 3 Regenerate **agents.md** from your updated rules document.
- 4 Use the **agents.md** file and Copilot (or your preferred AI) to generate **scripts/04_evaluation.R**. Run the script and check the output.

3. Output Table

- Now that we have extracted our metrics, we can begin to prepare the output table(s).
- First, we want a table with one row per evaluated `reference_date` and `horizon` combination, with columns for `reference_date`, `horizon`, `target_end_date`, `observed`, `WIS`, `AE`, and `coverage_95`.

Example Table: Sorted ascending by `reference_date`, then `horizon`. Metrics rounded to one decimal.

reference_date	horizon	target_end_date	observed	WIS	AE	coverage_95
2025-09-27	1	2025-10-04	976	332.2	174	1
2025-09-27	2	2025-10-11	1016	577	202	1
2025-09-27	3	2025-10-18	1119	754	99	1

File path to save to: ``output/data/04_evaluation/forecast_scores.csv`` (create the folder if absent).



Now it's your turn!

On your own, use the previous slides to specify rules to format the first of two output tables (Hint: your only task is to reformat the results we already have).

- 1 **Create** the `## 3. Output Table` section header under the `## 2. Scoring with scoringutils` header in **rules.md**.
- 2 **Write detailed rules** to create the first output table (**Hint:** refer to previous slide for format), sort by reference date and then horizon, round the metrics to one decimal, and print & save the table.
- 3 Regenerate **agents.md** and use it and Copilot (or your preferred AI) to generate **scripts/04_evaluation.R**. Run the script and check the output.

Expected Output

	reference_date	horizon	target_end_date	observed	WIS	MAE	coverage_95
1	2025-09-27	1	2025-10-04	976	332.2	174	1
2	2025-09-27	2	2025-10-11	1016	577	202	1
3	2025-09-27	3	2025-10-18	1119	754	99	1
4	2025-10-04	1	2025-10-11	1016	312.5	108	1
5	2025-10-04	2	2025-10-18	1119	532.3	269	1
6	2025-10-04	3	2025-10-25	1199	705.1	349	1
7	2025-10-11	1	2025-10-18	1119	319.9	76	1
8	2025-10-11	2	2025-10-25	1199	554.5	120	1
9	2025-10-11	3	2025-11-01	1358	737.6	279	1
10	2025-10-18	1	2025-10-25	1199	326.8	16	1
11	2025-10-18	2	2025-11-01	1358	573.8	118	1
12	2025-10-18	3	2025-11-08	1787	774	547	1
13	2025-10-25	1	2025-11-01	1358	331.8	146	1
14	2025-10-25	2	2025-11-08	1787	601.9	580	1
15	2025-10-25	3	2025-11-15	2487	859.1	1280	1
16	2025-11-01	1	2025-11-08	1787	359.3	320	1
17	2025-11-01	2	2025-11-15	2487	693.7	971	1
18	2025-11-01	3	2025-11-22	3584	1084.1	2068	1
19	2025-11-08	1	2025-11-15	2487	382.7	400	1
20							

4. Summary by Horizon

- **This is where per-forecast scores become forecast *skill*.** Averaging each horizon's scores across all its reference dates shows how accuracy changes with lead time.
- **We want one row per horizon:** the mean of `WIS`, `MAE`, and `coverage_95`, the min/max of `WIS` and `MAE`, and `n` (# reference dates). The mean of the 0/1 coverage *is* the empirical 95% coverage rate

Example Table: sorted ascending by horizon. `WIS_*` and `MAE_*` rounded to one decimal, `coverage_95_mean` to two.

horizon	n	WIS_mean	WIS_min	WIS_max	MAE_mean	MAE_min	MAE_max	coverage_95_mean
1	34	245.8	88.4	512.6	168.3	42.0	388.0	0.92
2	34	431.5	190.2	803.1	236.7	95.0	511.0	0.82
3	34	612.9	268.7	1094.4	331.4	99.0	702.0	0.70

File path to save to: `output/data/04\_evaluation/forecast\_scores\_by\_horizon.csv` (create the folder if absent).



Now it's your turn!

On your own, use the previous slides to specify rules to format the second of two output tables (**Hint**: your only task is to reformat the results we already have).

1

Create the `## 4. Summary by Horizon` section header under the `## 3. Output Table` header in **rules.md**.

2

Write detailed rules to create the second output table (**Hint**: refer to previous slide for format). Group by `horizon`, compute the mean of `WIS`, `MAE`, and `coverage_95` plus the min/max of `WIS` and `MAE`, sort ascending by `horizon`, round `WIS/MAE` to one decimal and `coverage_95_mean` to two, then print & save the table.

3

Regenerate `agents.md` and use it and Copilot (or your preferred AI) to generate **scripts/04_evaluation.R**. Run the script and check the output.

Expected Output

```
1 horizon,n,WIS_mean,WIS_min,WIS_max,MAE_mean,MAE_min,MAE_max,coverage_95_mean
2 1,34,1529.8,312.5,10129.1,1954.2,16,11505,0.88
3 2,34,3493.9,532.3,23364.3,4407.7,34,26148,0.85
4 3,34,4862.8,705.1,36917.7,6102.2,99,41058,0.85
```